Второй мозг из переписок с Claude: полный мануал

Как превратить 8 месяцев переписок с Claude AI в граф знаний с 1,649 заметками и 8,309 связями за 30 минут.

Автор: Алекс Манье · ReveluxAI · t.me/aidev_alexmagnier

Содержание

→ 01 — Зачем это нужно

→ 02 — Выгрузка данных из Claude

→ 03 — Установка инструментов

→ 04 — Скрипт конвертации

→ 05 — Настройка и кастомизация

→ 06 — Запуск скрипта

→ 07 — Открытие в Obsidian + граф знаний

→ 08 — Навигация и плагины

→ 09 — Запросы Dataview

→ 10 — Продвинутое: Graphify + RAG

→ 11 — FAQ

→ 12 — Автор и ресурсы

01 — Зачем это нужно

Проблема

Ты работаешь с Claude каждый день. Спрашиваешь, решаешь задачи, пишешь спецификации, находишь архитектурные решения. Через 3–6 месяцев у тебя сотни переписок. Но:

→ Нет поиска по всем перепискам одновременно

→ Нельзя увидеть связи между темами

→ Решение трёхмесячной давности — потеряно навсегда

→ Нельзя переиспользовать наработки

Решение

Второй мозг — это Obsidian vault, в который автоматически конвертируются все твои переписки с Claude. Каждая переписка = отдельная заметка. Каждый проект = отдельная заметка. Между ними — `[[wikilinks]]`, которые Obsidian визуализирует как интерактивный граф знаний.

Что ты получишь

На моём примере (8 месяцев работы с Claude):

Метрика	Значение
Заметок	1,649
Wikilinks (связей)	8,309
Conversations	326
Projects	67
Code Snippets	1,111
Memories	33
Concept nodes	24
MOC (навигация)	9

Всё это генерируется автоматически за 2–3 минуты одним скриптом.

Для кого

→ Разработчики, использующие Claude Code

→ Предприниматели, ведущие проекты через Claude AI

→ Маркетологи, генерирующие контент с AI

→ Все, кто хочет превратить хаос переписок в управляемую базу знаний

02 — Выгрузка данных из Claude

Шаг 1: Экспорт из Claude AI (web)

1. Зайди на claude.ai
2. Открой **Settings** (иконка шестерёнки)
3. Прокрутить вниз до **"Account"**
4. Нажми **"Export Data"**
5. Подтверди email — на почту придёт ссылка на скачивание
6. Скачай архив (может быть 100–500 МБ в зависимости от объёма переписок)

Шаг 2: Распаковка архива

После скачивания ты получишь ZIP-архив. Распакуй его в удобное место:

```
C:\Users\ТвойЮзер\Desktop\BRAIN_CLAUDE\
```

Внутри будут 4 файла:

Файл	Размер	Что содержит
`conversations.json`	10–500 МБ	Все переписки с Claude (сообщения, attachments)
`projects.json`	1–20 МБ	Все Claude Projects (настройки, загруженные документы)
`memories.json`	50–200 КБ	Память Claude о тебе (глобальная + проектная)
`users.json`	<1 КБ	Твой профиль (имя, email)

> ⚠️ ВАЖНО: `conversations.json` может быть очень большим (у меня — 293 МБ). Не пытайся открыть его в блокноте — он зависнет. Наш скрипт обрабатывает его по частям.

Шаг 3: Данные Claude Code (опционально)

Если ты работал с Claude Code (VS Code extension), то у тебя есть дополнительные данные локально:

Windows:

```
C:\Users\ТвойЮзер\.claude\projects\
```

Mac/Linux:

```
~/.claude/projects/
```

Внутри — папки для каждого проекта, в которых лежат `.jsonl` файлы (по одному на сессию). Скрипт умеет их читать тоже.

Этот шаг опциональный. Даже без данных Claude Code, граф из web-переписок будет полноценным.

Шаг 4: Создание рабочей папки

Создай папку для будущего Obsidian vault:

```
C:\Users\ТвойЮзер\Desktop\MY_BRAIN\
```

Итоговая структура файлов:

```
Desktop/
├── BRAIN_CLAUDE/           ← сюда распаковал export
│   ├── conversations.json
│   ├── projects.json
│   ├── memories.json
│   └── users.json
│
└── MY_BRAIN/               ← сюда скрипт создаст vault
    └── (пока пусто)
```

03 — Установка инструментов

Obsidian (бесплатно)

1. Скачай с obsidian.md
2. Установи (Windows/Mac/Linux)
3. Пока не открывай — vault ещё не готов

Python 3.11+ (бесплатно)

Проверь, установлен ли Python:

```
python --version
```

Если видишь `Python 3.11.x` или выше — ты в порядке.

Если нет — скачай с python.org:

→ Бери версию **LTS** (последняя стабильная)

→ При установке поставь галочку **"Add to PATH"**

Зависимости (1 минута)

Открой терминал и установи две библиотеки:

```
pip install pyyaml unidecode
```

Что они делают:

→ **pyyaml** — для работы с YAML-frontmatter в markdown-заметках

→ **unidecode** — для транслитерации русских названий в безопасные имена файлов

Если у тебя Mac/Linux и `pip` не работает — попробуй `pip3 install pyyaml unidecode`

04 — Скрипт конвертации

Создание файла скрипта

Создай файл `build_brain.py` в папке `BRAIN_CLAUDE/` (рядом с JSON-файлами):

```
BRAIN_CLAUDE/
├── conversations.json
├── projects.json
├── memories.json
├── users.json
└── build_brain.py    ← создай этот файл
```

Код скрипта

Скопируй код ниже целиком в `build_brain.py`.

> ⚠️ ВАЖНО: Перед запуском отредактируй 3 строки в секции CONFIG (указаны комментариями `# ← ИЗМЕНИ`)

```python
#!/usr/bin/env python3
"""
Second Brain Builder
Конвертирует экспорт Claude AI в Obsidian-совместимый граф знаний.
"""

import json
import re
import os
import sys
from pathlib import Path
from datetime import datetime
from collections import defaultdict, Counter

from unidecode import unidecode

# ==============================================================================
# CONFIG — ИЗМЕНИ ЭТИ 3 СТРОКИ ПОД СВОИ ПУТИ
# ==============================================================================

# Папка, где лежат conversations.json, projects.json и т.д.
SOURCE_DIR = Path(r"C:\Users\ТвойЮзер\Desktop\BRAIN_CLAUDE")  # ← ИЗМЕНИ

# Папка .claude (для данных Claude Code). Если не используешь Claude Code — оставь как е
CLAUDE_CODE_DIR = Path.home() / ".claude"  # обычно не нужно менять

# Папка, куда создастся Obsidian vault
VAULT_DIR = Path(r"C:\Users\ТвойЮзер\Desktop\MY_BRAIN")  # ← ИЗМЕНИ

# Файлы источников (не трогай, если не переименовывал)
CONVERSATIONS_FILE = SOURCE_DIR / "conversations.json"
PROJECTS_FILE = SOURCE_DIR / "projects.json"
MEMORIES_FILE = SOURCE_DIR / "memories.json"
USERS_FILE = SOURCE_DIR / "users.json"

MIN_CODE_LINES = 5  # Минимум строк для извлечения code snippets

# ==============================================================================
# ДОМЕНЫ — можешь добавить свои ключевые слова для автоматической классификации
# ==============================================================================

DOMAIN_KEYWORDS = {
    "marketing": [
        "маркетинг", "marketing", "копирайтинг", "copywriting",
        "контент", "content", "воронк", "funnel", "конверси",
```

```
        "лендинг", "landing", "seo", "smm", "реклам", "instagram",
        "telegram", "youtube",
    ],
    "development": [
        "разработк", "development", "код", "code", "api", "supabase",
        "react", "next.js", "typescript", "javascript", "python",
        "database", "архитектур", "deploy", "git", "webhook",
        "frontend", "backend", "vercel", "tailwind", "sql",
    ],
    "ai-tools": [
        "ai", "ии", "claude", "chatgpt", "gpt", "openai", "prompt",
        "промпт", "llm", "нейросет", "автоматизац", "n8n",
    ],
    "business": [
        "бизнес", "business", "клиент", "client", "стратеги",
        "strategy", "стартап", "startup", "агентств", "agency",
        "финанс", "инвестиц",
    ],
    "education": [
        "курс", "course", "обучени", "урок", "lesson",
        "вебинар", "webinar", "менторств",
    ],
}

# Известные концепции — станут [[wikilinks]] в тексте заметок
KNOWN_CONCEPTS = {
    "Supabase": "Supabase",
    "Claude": "Claude",
    "Claude Code": "Claude-Code",
    "Vercel": "Vercel",
    "Telegram": "Telegram",
    "YouTube": "YouTube",
    "Next.js": "Next-js",
    "TypeScript": "TypeScript",
    "Tailwind": "Tailwind",
    "Obsidian": "Obsidian",
}


# ============================================================================
# UTILITIES
# ============================================================================

def slugify(text, max_len=80):
    """Превращает текст в безопасное имя файла."""
    if not text:
```

```
        return "untitled"
    slug = unidecode(text).lower()
    slug = re.sub(r'[^\w\s-]', '', slug)
    slug = re.sub(r'[-\s]+', '-', slug).strip('-')
    return slug[:max_len] if slug else "untitled"


def detect_language(text):
    """Определяет язык текста (ru/en/mixed)."""
    if not text:
        return "en"
    cyrillic = len(re.findall(r'[а-яА-ЯёЁ]', text))
    latin = len(re.findall(r'[a-zA-Z]', text))
    total = cyrillic + latin
    if total == 0:
        return "en"
    ratio = cyrillic / total
    if ratio > 0.6:
        return "ru"
    elif ratio < 0.2:
        return "en"
    return "mixed"


def extract_code_blocks(text):
    """Извлекает блоки кода с >= MIN_CODE_LINES строк."""
    pattern = r'(\w*)\n(.*?)'
    blocks = []
    for match in re.finditer(pattern, text, re.DOTALL):
        lang = match.group(1) or "other"
        code = match.group(2).strip()
        if code.count('\n') + 1 >= MIN_CODE_LINES:
            start = max(0, match.start() - 200)
            context = text[start:match.start()].strip().split('\n')[-2:]
            blocks.append({
                'lang': lang.lower(),
                'code': code,
                'context': '\n'.join(context).strip(),
            })
    return blocks


def auto_tag_domains(text):
    """Автоматически определяет домены (теги) из текста."""
    if not text:
        return []
```

```
    text_lower = text.lower()
    tags = []
    for domain, keywords in DOMAIN_KEYWORDS.items():
        for kw in keywords:
            if kw in text_lower:
                tags.append(f"domain/{domain}")
                break
    return tags


def format_date(iso_str):
    """ISO 8601 → YYYY-MM-DD."""
    return iso_str[:10] if iso_str else "unknown"


def format_datetime(iso_str):
    """ISO 8601 → YYYY-MM-DD HH:MM."""
    if not iso_str:
        return ""
    try:
        dt = datetime.fromisoformat(iso_str.replace('Z', '+00:00'))
        return dt.strftime('%Y-%m-%d %H:%M')
    except Exception:
        return iso_str[:16] if iso_str else ""


def get_month_folder(iso_str):
    """ISO date → YYYY-MM для папки."""
    return iso_str[:7] if iso_str else "unknown"


def write_note(filepath, frontmatter, body):
    """Записывает markdown-заметку с YAML-frontmatter."""
    filepath.parent.mkdir(parents=True, exist_ok=True)
    lines = ["---"]
    for key, value in frontmatter.items():
        if isinstance(value, list):
            if not value:
                lines.append(f"{key}: []")
            else:
                lines.append(f"{key}:")
                for item in value:
                    lines.append(f"  - {item}")
        elif isinstance(value, bool):
            lines.append(f"{key}: {'true' if value else 'false'}")
        elif isinstance(value, int):
```

```
            lines.append(f"{key}: {value}")
        elif value is None:
            lines.append(f"{key}: ")
        else:
            val_str = str(value)
            if ':' in val_str or '"' in val_str or "'" in val_str:
                val_str = val_str.replace('"', '\\"')
                lines.append(f'{key}: "{val_str}"')
            else:
                lines.append(f"{key}: {val_str}")
    lines.append("---")
    lines.append("")
    lines.append(body)
    with open(filepath, 'w', encoding='utf-8') as f:
        f.write('\n'.join(lines))


def inject_wikilinks(text, concept_registry):
    """Заменяет упоминания концепций на [[wikilinks]]."""
    if not text:
        return text
    sorted_concepts = sorted(concept_registry.keys(), key=len, reverse=True)
    for concept_name in sorted_concepts:
        link_target = concept_registry[concept_name]
        pattern = r'(? 2:
            concept_registry[name] = slug

        combined_text = f"{name} {description} {prompt_template[:500]}"
        domain_tags = auto_tag_domains(combined_text)
        tags = ["source/claude-web", "type/project", f"project/{slug}"] + domain_tags

        body_parts = [f"# {name}\n"]
        if description:
            body_parts.append(f"## Description\n\n{description}\n")
        if prompt_template:
            prompt_display = prompt_template if len(prompt_template) < 5000 else prompt_
            body_parts.append(f"## System Prompt\n\n> {prompt_display.replace(chr(10), c
        if docs:
            body_parts.append("## Documents\n")
            for doc in docs:
                doc_filename = doc.get('filename', 'untitled')
                doc_slug = slugify(doc_filename.rsplit('.', 1)[0] if '.' in doc_filename
                body_parts.append(f"- [[{slug}--{doc_slug}|{doc_filename}]]")
            body_parts.append("")

        frontmatter = {
```

```
            'uuid': uuid, 'type': 'project', 'source': 'claude-web',
            'title': name, 'created': created, 'updated': updated,
            'doc_count': len(docs), 'tags': tags,
        }
        write_note(VAULT_DIR / "Projects" / "Web" / f"{slug}.md", frontmatter, '\n'.join
        project_notes += 1

        for doc in docs:
            doc_filename = doc.get('filename', 'untitled')
            doc_content = doc.get('content', '') or ''
            doc_created = format_date(doc.get('created_at', ''))
            doc_slug = slugify(doc_filename.rsplit('.', 1)[0] if '.' in doc_filename els
            ext = doc_filename.rsplit('.', 1)[1].lower() if '.' in doc_filename else 'tx

            doc_frontmatter = {
                'type': 'project-doc', 'project': f"[[{slug}]]",
                'filename': doc_filename, 'created': doc_created,
                'tags': ["source/project-doc", f"project/{slug}", f"format/{ext}"],
            }
            doc_body = f"# {doc_filename}\n\n*Project: [[{slug}|{name}]]*\n\n{doc_conten
            doc_dir = VAULT_DIR / "Project-Docs" / slug
            write_note(doc_dir / f"{slug}--{doc_slug}.md", doc_frontmatter, doc_body)
            doc_notes += 1

    print(f"    Created {project_notes} project notes, {doc_notes} doc notes")
    return project_map


def parse_memories(project_map):
    """Парсит memories.json → Memory notes."""
    print("  Parsing memories.json...")
    with open(MEMORIES_FILE, 'r', encoding='utf-8') as f:
        memories = json.load(f)

    mem = memories[0]
    notes = 0

    global_memory = mem.get('conversations_memory', '')
    if global_memory:
        write_note(VAULT_DIR / "Memories" / "global-memory.md",
                   {'type': 'memory', 'scope': 'global', 'tags': ["source/memory", "type
                   f"# Global Memory\n\n{global_memory}")
        notes += 1

    project_memories = mem.get('project_memories', {})
    if isinstance(project_memories, dict):
```

```
        for pm_uuid, pm_content in project_memories.items():
            proj_info = project_map.get(pm_uuid, {})
            proj_name = proj_info.get('name', pm_uuid[:8])
            proj_slug = proj_info.get('slug', slugify(pm_uuid[:8]))

            body = f"# Memory: {proj_name}\n\n*Project: [[{proj_slug}|{proj_name}]]*\n\n
            body += pm_content if isinstance(pm_content, str) else json.dumps(pm_content

            write_note(VAULT_DIR / "Memories" / f"{proj_slug}-memory.md",
                       {'type': 'memory', 'scope': 'project', 'project': f"[[{proj_slug}
                        'tags': ["source/memory", "type/memory", f"project/{proj_slug}"]
                       body)
            notes += 1

    print(f"    Created {notes} memory notes")


def process_message_content(content_items):
    """Конвертирует content array из сообщения в markdown."""
    parts = []
    for item in content_items:
        item_type = item.get('type', '')
        if item_type == 'text':
            text = item.get('text', '')
            if text and text.strip():
                parts.append(text.strip())
        elif item_type == 'tool_use':
            tool_name = item.get('name', 'unknown')
            parts.append(f"> *[Tool: {tool_name}]*")
    return '\n\n'.join(parts)


def parse_conversations(project_map, concept_registry):
    """Парсит conversations.json → Conversation notes + Code snippets."""
    print("  Parsing conversations.json (это может занять 1-2 минуты)...")

    with open(CONVERSATIONS_FILE, 'r', encoding='utf-8') as f:
        conversations = json.load(f)

    print(f"    Загружено {len(conversations)} conversations")

    conv_notes = 0
    code_snippets = 0
    all_conv_metadata = []
    seen_slugs = defaultdict(int)
```

```
    for i, conv in enumerate(conversations):
        messages = conv.get('chat_messages', [])
        if not messages:
            continue

        name = conv.get('name', '') or f"Untitled-{conv['uuid'][:8]}"
        summary = conv.get('summary', '') or ''
        created = conv.get('created_at', '')
        updated = conv.get('updated_at', '')
        date_str = format_date(created)
        month_folder = get_month_folder(created)

        base_slug = f"{date_str}_{slugify(name, 60)}"
        seen_slugs[base_slug] += 1
        slug = f"{base_slug}-{seen_slugs[base_slug]}" if seen_slugs[base_slug] > 1 else

        first_human_text = ''
        for msg in messages:
            if msg.get('sender') == 'human':
                first_human_text = msg.get('text', '') or ''
                break
        lang = detect_language(first_human_text[:300])

        tag_text = f"{name} {summary} {first_human_text[:500]}"
        domain_tags = auto_tag_domains(tag_text)

        matched_projects = match_conversation_to_project(name, summary, first_human_text
        project_tags = [f"project/{project_map[pid]['slug']}" for pid in matched_project

        has_code = False
        has_attachments = False
        full_text_parts = []
        body_parts = [f"# {name}\n"]

        if summary:
            body_parts.append(f"> **Summary:** {summary}\n")

        body_parts.append("## Conversation\n")

        for msg in messages:
            sender = msg.get('sender', 'unknown')
            msg_time = format_datetime(msg.get('created_at', ''))
            content_items = msg.get('content', [])
            text = msg.get('text', '') or ''
            attachments = msg.get('attachments', [])
```

```
        processed = process_message_content(content_items) if content_items else tex
        if not processed.strip():
            continue

        sender_label = "Human" if sender == "human" else "Assistant"
        body_parts.append(f"### {sender_label} ({msg_time})\n")
        body_parts.append(processed)
        body_parts.append("")
        full_text_parts.append(processed)

        if '' in processed:
            has_code = True
        if attachments:
            has_attachments = True
            for att in attachments:
                att_name = att.get('file_name', 'attachment')
                att_content = att.get('extracted_content', '')
                if att_content:
                    body_parts.append(f"**Attachment: {att_name}**\n")
                    body_parts.append(f"{att_content[:3000]}\n" if len(att_content)

    tags = ["source/claude-web", "type/conversation"] + domain_tags + project_tags
    if has_code:
        tags.append("has-code")

    frontmatter = {
        'uuid': conv['uuid'], 'type': 'conversation', 'title': name,
        'created': date_str, 'updated': format_date(updated),
        'message_count': len(messages), 'has_code': has_code,
        'has_attachments': has_attachments, 'language': lang, 'tags': tags,
    }

    body_text = inject_wikilinks('\n'.join(body_parts), concept_registry)
    write_note(VAULT_DIR / "Conversations" / month_folder / f"{slug}.md", frontmatte
    conv_notes += 1

    all_conv_metadata.append({
        'uuid': conv['uuid'], 'name': name, 'slug': slug,
        'date': date_str, 'month': month_folder, 'summary': summary,
        'matched_projects': matched_projects, 'domain_tags': domain_tags,
        'message_count': len(messages),
    })

    full_text = '\n'.join(full_text_parts)
    code_blocks = extract_code_blocks(full_text)
    for j, block in enumerate(code_blocks):
```

```
            lang_folder = block['lang'] if block['lang'] in ['javascript', 'typescript',
            snippet_slug = f"{slug}--snippet-{j+1}"
            snippet_body = f"# Code from: {name}\n\n"
            if block['context']:
                snippet_body += f"> {block['context']}\n\n"
            snippet_body += f"{block['lang']}\n{block['code']}\n\n\n"
            snippet_body += f"## Source\n\n- [[{slug}|{name}]]\n"

            write_note(VAULT_DIR / "Code-Snippets" / lang_folder / f"{snippet_slug}.md",
                       {'type': 'code-snippet', 'language': block['lang'],
                        'source_conversation': f"[[{slug}]]", 'created': date_str,
                        'tags': ["source/code-extract", "type/code-snippet", f"lang/{blo
                       snippet_body)
            code_snippets += 1

        if (i + 1) % 50 == 0:
            print(f"    Processed {i+1}/{len(conversations)}...")

    print(f"    Created {conv_notes} conversation notes, {code_snippets} code snippets")
    return all_conv_metadata


def parse_claude_code(project_map, concept_registry):
    """Парсит Claude Code JSONL-сессии."""
    print("  Parsing Claude Code sessions...")
    claude_projects_dir = CLAUDE_CODE_DIR / "projects"
    if not claude_projects_dir.exists():
        print("    Папка Claude Code не найдена, пропускаю.")
        return []

    code_session_metadata = []
    sessions_created = 0

    for proj_dir in claude_projects_dir.iterdir():
        if not proj_dir.is_dir():
            continue

        dir_name = proj_dir.name
        proj_name = dir_name
        if 'Desktop' in dir_name:
            idx = dir_name.find('Desktop-')
            if idx >= 0:
                proj_name = dir_name[idx + 8:].replace('-', ' ').strip()
        elif dir_name.lower().startswith('c--users'):
            proj_name = re.sub(r'^[cC]--Users-\w+-?', '', dir_name).replace('-', ' ').st
        if not proj_name:
```

```
        proj_name = dir_name

    proj_slug = slugify(proj_name)

    code_proj_body = f"# {proj_name} (Claude Code)\n\n*Source: Claude Code*\n\n## Se

    session_files = [f for f in proj_dir.iterdir() if f.suffix == '.jsonl']

    for sf in session_files:
        session_id = sf.stem[:8]
        try:
            messages = parse_jsonl_session(sf)
        except Exception as e:
            print(f"    Warning: Failed to parse {sf}: {e}")
            continue
        if not messages:
            continue

        session_slug = f"code-{proj_slug}-{session_id}"
        first_date = messages[0].get('date', 'unknown')

        session_body = f"# Code Session: {proj_name}\n\n*Session ID: {sf.stem}*\n\n#
        for msg in messages:
            role_label = "Human" if msg['role'] == "user" else "Assistant"
            session_body += f"### {role_label}\n\n{msg['text']}\n\n"

        session_body = inject_wikilinks(session_body, concept_registry)
        domain_tags = auto_tag_domains(session_body[:2000])

        write_note(VAULT_DIR / "Projects" / "Code" / f"{session_slug}.md",
                   {'type': 'code-session', 'source': 'claude-code',
                    'project': f"[[{proj_slug}]]", 'session_id': sf.stem,
                    'created': first_date, 'message_count': len(messages),
                    'tags': ["source/claude-code", "type/code-session", f"project/{p
                   session_body)
        sessions_created += 1
        code_proj_body += f"- [[{session_slug}|Session {session_id}]] ({len(messages
        code_session_metadata.append({'slug': session_slug, 'project': proj_name, 'd

    write_note(VAULT_DIR / "Projects" / "Code" / f"{proj_slug}.md",
               {'type': 'project', 'source': 'claude-code', 'title': proj_name,
                'tags': ["source/claude-code", "type/project", f"project/{proj_slug}
               code_proj_body)

print(f"    Created {sessions_created} code session notes")
return code_session_metadata
```

```
def parse_jsonl_session(filepath):
    """Парсит JSONL-файл Claude Code сессии."""
    messages = []
    with open(filepath, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            try:
                entry = json.loads(line)
            except json.JSONDecodeError:
                continue

            entry_type = entry.get('type', '')
            if entry_type in ('user', 'assistant'):
                message = entry.get('message', {})
                content = message.get('content', '')
                text = ''
                if isinstance(content, str):
                    text = content
                elif isinstance(content, list):
                    text_parts = []
                    for item in content:
                        if isinstance(item, dict) and item.get('type') == 'text':
                            text_parts.append(item.get('text', ''))
                        elif isinstance(item, dict) and item.get('type') == 'tool_use':
                            text_parts.append(f"> *[Tool: {item.get('name', 'unknown')}]
                        elif isinstance(item, str):
                            text_parts.append(item)
                    text = '\n'.join(text_parts)
                if text.strip():
                    timestamp = entry.get('timestamp', '')
                    messages.append({
                        'role': entry_type, 'text': text.strip(),
                        'date': format_date(timestamp) if timestamp else 'unknown',
                    })
    return messages


# ============================================================================
# LINKERS
# ============================================================================

def match_conversation_to_project(conv_name, summary, first_msg, project_map):
```

```
    """Сопоставляет переписку с проектом по ключевым словам."""
    matched = []
    search_text = f"{conv_name} {summary} {first_msg[:500]}".lower()

    for uuid, info in project_map.items():
        proj_name_lower = info['name'].lower()
        if len(proj_name_lower) > 3 and proj_name_lower in search_text:
            matched.append(uuid)
            continue
        matching_kw = sum(1 for kw in info['keywords'] if len(kw) >= 5 and kw in search_
        if matching_kw >= 2:
            matched.append(uuid)
    return matched


def find_related_conversations(all_conv_metadata):
    """Находит связанные переписки по похожим названиям."""
    relations = defaultdict(set)
    for i, c1 in enumerate(all_conv_metadata):
        name1_words = set(re.findall(r'\w{4,}', c1['name'].lower()))
        for j, c2 in enumerate(all_conv_metadata):
            if i >= j:
                continue
            name2_words = set(re.findall(r'\w{4,}', c2['name'].lower()))
            shared = name1_words & name2_words
            if len(shared) >= 3:
                relations[c1['slug']].add(c2['slug'])
                relations[c2['slug']].add(c1['slug'])
    return relations


# ============================================================================
# GENERATORS
# ============================================================================

def generate_concept_notes(concept_registry, all_conv_metadata):
    """Создаёт ноды-концепции (Supabase, Claude, etc.)."""
    print("  Generating concept notes...")
    created = 0
    concept_convs = defaultdict(list)
    for conv in all_conv_metadata:
        text = f"{conv['name']} {conv.get('summary', '')}".lower()
        for concept_name, link_target in concept_registry.items():
            if concept_name.lower() in text:
                concept_convs[link_target].append(conv)
```

```
    for link_target, convs in concept_convs.items():
        if len(convs) < 2:
            continue
        body = f"# {link_target}\n\n*Mentioned in {len(convs)} conversations*\n\n## Conv
        for c in convs[:50]:
            body += f"- [[{c['slug']}|{c['name']}]] ({c['date']})\n"
        write_note(VAULT_DIR / "Concepts" / f"{slugify(link_target)}.md",
                   {'type': 'concept', 'mention_count': len(convs), 'tags': ["type/conce
                   body)
        created += 1
    print(f"    Created {created} concept notes")


def generate_moc_notes(all_conv_metadata):
    """Создаёт Maps of Content по каждому домену."""
    print("  Generating MOC notes...")
    domain_convs = defaultdict(list)
    for conv in all_conv_metadata:
        for tag in conv.get('domain_tags', []):
            domain = tag.replace('domain/', '')
            domain_convs[domain].append(conv)

    for domain, convs in domain_convs.items():
        domain_title = domain.replace('-', ' ').title()
        body = f"# {domain_title}\n\n*{len(convs)} conversations*\n\n"
        month_groups = defaultdict(list)
        for c in convs:
            month_groups[c['month']].append(c)
        for month in sorted(month_groups.keys(), reverse=True):
            body += f"## {month}\n\n"
            for c in sorted(month_groups[month], key=lambda x: x['date'], reverse=True):
                body += f"- [[{c['slug']}|{c['name']}]] ({c['message_count']} msgs)\n"
            body += "\n"
        write_note(VAULT_DIR / "MOC" / f"{domain_title}.md",
                   {'type': 'moc', 'domain': domain, 'conversation_count': len(convs),
                    'tags': ["type/moc", f"domain/{domain}"]},
                   body)
    print(f"    Created {len(domain_convs)} MOC notes")


def generate_index(all_conv_metadata, project_map):
    """Создаёт _INDEX.md — главную навигацию."""
    print("  Generating index notes...")
    total_convs = len(all_conv_metadata)
    total_msgs = sum(c['message_count'] for c in all_conv_metadata)
    dates = [c['date'] for c in all_conv_metadata if c['date'] != 'unknown']
```

```
    date_range = f"{min(dates)} to {max(dates)}" if dates else "unknown"

    body = f"""# Second Brain\n\n> {total_convs} conversations | {len(project_map)} proj
    for domain in sorted(DOMAIN_KEYWORDS.keys()):
        title = domain.replace('-', ' ').title()
        body += f"- [[{title}]]\n"

    body += "\n## Projects\n\n"
    for uuid, info in sorted(project_map.items(), key=lambda x: x[1]['name']):
        body += f"- [[{info['slug']}|{info['name']}]]\n"

    body += "\n## Recent Conversations\n\n"
    recent = sorted(all_conv_metadata, key=lambda x: x['date'], reverse=True)[:30]
    for c in recent:
        body += f"- [[{c['slug']}|{c['name']}]] ({c['date']}, {c['message_count']} msgs)

    write_note(VAULT_DIR / "_INDEX.md", {'type': 'index', 'tags': ["type/moc"]}, body)

    timeline_body = "# Timeline\n\n"
    month_groups = defaultdict(list)
    for c in all_conv_metadata:
        month_groups[c['month']].append(c)
    for month in sorted(month_groups.keys(), reverse=True):
        convs = sorted(month_groups[month], key=lambda x: x['date'], reverse=True)
        timeline_body += f"## {month} ({len(convs)} conversations)\n\n"
        for c in convs:
            timeline_body += f"- **{c['date']}** [[{c['slug']}|{c['name']}]] ({c['messag
        timeline_body += "\n"
    write_note(VAULT_DIR / "_TIMELINE.md", {'type': 'timeline', 'tags': ["type/moc"]}, t

    tag_body = "# Tag Index\n\n## Domains\n\n"
    tag_counts = Counter()
    for c in all_conv_metadata:
        for tag in c.get('domain_tags', []):
            tag_counts[tag] += 1
    for tag, count in sorted(tag_counts.items(), key=lambda x: -x[1]):
        domain = tag.replace('domain/', '')
        tag_body += f"- `{tag}` ({count} conversations)\n"
    write_note(VAULT_DIR / "_TAG_INDEX.md", {'type': 'tag-index', 'tags': ["type/moc"]},


# ==============================================================================
# MAIN
# ==============================================================================

def main():
```

```
    print("=" * 60)
    print("  SECOND BRAIN BUILDER")
    print("  Claude exports → Obsidian Knowledge Graph")
    print("=" * 60)
    print()

    VAULT_DIR.mkdir(parents=True, exist_ok=True)
    concept_registry = dict(KNOWN_CONCEPTS)

    print("[1/5] Users")
    user_info = parse_users()
    print(f"    User: {user_info.get('full_name', 'unknown')}")
    print()

    print("[2/5] Projects & Memories")
    project_map = parse_projects(concept_registry)
    parse_memories(project_map)
    print()

    print("[3/5] Conversations")
    all_conv_metadata = parse_conversations(project_map, concept_registry)
    print()

    print("[4/5] Claude Code Sessions")
    parse_claude_code(project_map, concept_registry)
    print()

    print("[5/5] Cross-linking & Generation")
    relations = find_related_conversations(all_conv_metadata)
    print(f"    Found {sum(len(v) for v in relations.values()) // 2} conversation pairs"
    generate_concept_notes(concept_registry, all_conv_metadata)
    generate_moc_notes(all_conv_metadata)
    generate_index(all_conv_metadata, project_map)
    print()

    total_files = sum(1 for root, _, files in os.walk(VAULT_DIR) for f in files if f.end

    print("=" * 60)
    print("  READY!")
    print("=" * 60)
    print(f"  Total markdown notes: {total_files}")
    print(f"  Vault: {VAULT_DIR}")
    print()
    print("  Next steps:")
    print("  1. Open Obsidian → File → Open Vault → select your vault folder")
    print("  2. Enable Graph View (Ctrl+G)")
```

```
    print("  3. Install plugins: Dataview, Tag Wrangler, Graph Analysis")
    print()


if __name__ == '__main__':
    if sys.platform == 'win32':
        sys.stdout.reconfigure(encoding='utf-8')
        sys.stderr.reconfigure(encoding='utf-8')
    main()
```

05 — Настройка и кастомизация

Обязательная настройка: 3 строки

Перед запуском отредактируй **3 строки** в секции CONFIG:

```
SOURCE_DIR = Path(r"C:\Users\ТвойЮзер\Desktop\BRAIN_CLAUDE")  # путь к JSON-файлам
VAULT_DIR = Path(r"C:\Users\ТвойЮзер\Desktop\MY_BRAIN")       # куда создать vault
```

Mac/Linux пользователи: замените `Path(r"C:\...")` на `Path("/Users/yourname/Desktop/...")` или `Path.home() / "Desktop" / "MY_BRAIN"`

Добавить свои домены для классификации

Найди словарь `DOMAIN_KEYWORDS` и добавь свою категорию:

```
"design": [
    "дизайн", "design", "figma", "ui", "ux", "макет",
    "прототип", "wireframe",
],
```

Добавить свои концепции для wikilinks

Найди словарь `KNOWN_CONCEPTS` и добавь свои инструменты/проекты:

```
"Figma": "Figma",
"ChatGPT": "ChatGPT",
"Notion": "Notion",
"МойПроект": "moy-proekt",
```

Каждый добавленный концепт автоматически станет `[[wikilink]]` в тексте заметок, а Obsidian покажет его как узел на графе.

06 — Запуск скрипта

Открой терминал в папке `BRAIN_CLAUDE/` и запусти:

```
python build_brain.py
```

> ⚠️ ВАЖНО: На Windows если `python` не работает — попробуй `python3` или `py`

Ожидаемый вывод

```
============================================================
  SECOND BRAIN BUILDER
  Claude exports → Obsidian Knowledge Graph
============================================================

[1/5] Users
    User: Твоё Имя

[2/5] Projects & Memories
    Created 36 project notes, 67 doc notes
    Created 33 memory notes

[3/5] Conversations
    Загружено 339 conversations
    Processed 50/339...
    Processed 100/339...
    Created 326 conversation notes, 1111 code snippets

[4/5] Claude Code Sessions
    Created 21 code session notes

[5/5] Cross-linking & Generation
    Found 1274 conversation pairs
    Created 24 concept notes
    Created 9 MOC notes

============================================================
  READY!
============================================================
  Total markdown notes: 1649
  Vault: C:\Users\ТвойЮзер\Desktop\MY_BRAIN
```

Скрипт обрабатывает 300 МБ данных за 2–3 минуты. Если зависает — проверь, что пути в CONFIG указаны правильно.

07 — Открытие в Obsidian + граф знаний

Открытие vault

1. Запусти Obsidian
2. **File** → **Open Vault** (или "Open another vault")

3. Выбери **"Open folder as vault"**
4. Навигируй к папке `MY_BRAIN`
5. Нажми **Select Folder**

Первый взгляд на граф

1. Нажми `Ctrl+G` (Windows/Linux) или `Cmd+G` (Mac)
2. Откроется **Graph View** — визуальная карта всех заметок и связей

Что ты видишь:

→ **Крупные узлы** — концепции (Supabase, Claude, React) с множеством связей

→ **Кластеры** — группы переписок по темам (маркетинг, разработка, бизнес)

→ **Линии** — `[[wikilinks]]` между заметками

→ **Цвета** — можно настроить по тегам (domain/marketing = синий, domain/development = зелёный)

08 — Навигация и плагины

Навигация

Открой `_INDEX.md` — это твоя главная страница:

→ **Timeline** — хронологический список всех переписок по месяцам

→ **MOC по доменам** — Marketing, Development, AI Tools, Business, Education

→ **Projects** — все проекты из Claude Projects

Рекомендуемые плагины

В Obsidian: **Settings** → **Community plugins** → **Turn on** → **Browse**

Плагин	Зачем
Dataview	SQL-подобные запросы по заметкам. Пример: «покажи все переписки с тегом domain/development за март»
Tag Wrangler	Массовое управление тегами: переименование, слияние, удаление
Graph Analysis	Расширенный анализ графа: самые связанные ноды, кратчайший путь между концепциями

09 — Запросы Dataview

После установки плагина Dataview создай заметку и вставь нужный запрос.

Все переписки за последний месяц

```
dataview
TABLE message_count, created
FROM "Conversations"
WHERE type = "conversation"
SORT created DESC
LIMIT 20
```

Все проекты с документами

```
dataview
TABLE doc_count, created
FROM "Projects/Web"
WHERE doc_count > 0
SORT doc_count DESC
```

Code snippets на Python

```
dataview
LIST
FROM "Code-Snippets/python"
SORT created DESC
LIMIT 10
```

10 — Продвинутое: Graphify + RAG

Graphify — AI-кластеризация графа знаний

Graphify — инструмент, который анализирует vault через AI и:

- → Находит скрытые связи между концепциями
- → Кластеризует сообщества (community detection)
- → Генерирует `GRAPH_REPORT.md` с аудитом всех связей
- → Помечает каждую связь: EXTRACTED / INFERRED / AMBIGUOUS

Установка:

```
pip install graphifyy
```

Подключение к Claude Code:

```
graphify install --platform windows
```

Запуск:

```
/graphify .
# или с выводом в Obsidian:
/graphify . --obsidian
```

Результат:

- → `graphify-out/graph.json` — queryable knowledge graph
- → `graphify-out/graph.html` — интерактивная визуализация

→ `graphify-out/GRAPH_REPORT.md` — текстовый отчёт

→ `_COMMUNITY_*.md` — заметки-обзоры по кластерам (если --obsidian)

Запросы к графу:

```
graphify query "Какие проекты я строил с Supabase?"
graphify query "Связь между маркетингом и разработкой"
graphify query "What tools do I use most?"
```

> ⚠️ ВАЖНО: Graphify использует Claude для семантического анализа — это стоит токены. Для vault из 1,649 заметок рассчитывай на 50–100K input tokens.

RAG на Supabase pgvector (для продвинутых)

Следующий шаг — превратить vault в **векторную базу знаний**, чтобы задавать вопросы на естественном языке.

Архитектура:

```
Obsidian Vault (1,649 заметок)
    ↓ embed через Voyage AI
Supabase pgvector (векторная БД)
    ↓ query
API endpoint / Telegram-бот / Claude Code MCP
```

Что это даёт:

→ Задаёшь вопрос: «Как я решал проблему с RLS в прошлом месяце?»

→ Система находит 5 самых релевантных заметок по смыслу (не по ключевым словам)

→ Claude формирует ответ с цитатами из твоих же переписок

Это тема для отдельного гайда — если интересно, пиши в комментариях.

11 — FAQ

Скрипт выдаёт ошибку «ModuleNotFoundError: No module named 'unidecode'»

Установи зависимости: `pip install pyyaml unidecode`. На Mac/Linux попробуй `pip3`.

Файл conversations.json слишком большой — скрипт зависает

Скрипт обрабатывает файл целиком в память. Если у тебя менее 8 ГБ оперативки и файл > 500 МБ — закрой лишние программы перед запуском. Для файлов > 1 ГБ может потребоваться потоковый парсинг (напиши в Telegram, помогу с модификацией).

Могу ли я запустить скрипт повторно после новых переписок?

Да. Сделай новый экспорт из Claude, замени JSON-файлы и перезапусти скрипт. Он перезапишет весь vault.

А данные Claude Code обязательны?

Нет, это опционально. Скрипт автоматически пропустит этот шаг, если папка `~/.claude/projects/` не существует.

Можно ли добавить свои заметки в vault?

Да. Создавай .md файлы где угодно внутри vault. Используй `[[wikilinks]]` для связей с существующими заметками. Они появятся на графе.

Скрипт поддерживает ChatGPT / другие AI?

Пока нет. Но формат скрипта расширяемый — при наличии JSON-экспорта из другого AI-чата можно добавить новый парсер.

12 — Автор и ресурсы

Алекс Манье — основатель AI-агентства ReveluxAI. Собирает production-grade SaaS проекты за 7–10 дней без опыта программирования. Вся разработка — через Claude Code.

Курс **«AI-Архитектор»** — 30 уроков за 30 дней. Научишься строить полноценные SaaS-проекты с Claude Code: от спецификации до деплоя. Spec-driven разработка, Supabase, Vercel,

интеграции — всё, что используется в реальном продакшне.

→ Telegram-канал: t.me/aidev_alexmagnier

→ Instagram: @alex_magnier

→ Предзапись на курс: alexmagnier.ru

> *Нашёл полезным? Перешли другу-разработчику, который тоже тонет в переписках с Claude.*